Notes for C++

Section 8 : Characters and Strings

Characters

Examples of characters: `'A'  'h'  '+'  ' '`. Note that each character is surrounded by single quotes. This is especially important for the space character, otherwise we would not see it! Here is a program that uses a character variable:

```
#include <iostream>
using namespace std;

char k;

int main() {
  k = 'A';
  cout << "The character is " << k << endl;
}
```

ASCII codes

Computers do not store characters – they only store numbers. In order to store characters they have to first be converted to numbers. There is an international set of codes (the ASCII code) that tell us which integer value each character will be converted to. You can find more information here: https://theasciicode.com.ar/

You should know the following ASCII codes:

`'A' is 65.  'a' is 97.  '0' is 48.  ' ' is 32.`

The first three start a sequence, e.g. `'B' is 66` and `'d' is 100` and `'2' is 50`.

Note that we often store digits (or numbers) as characters. For example, phone numbers and ID numbers look like numbers but are not used as numbers. For example, we do not add two phone numbers together.

We can load integer values (or ASCII codes) directly into character variables and we can do arithmetic operations on the integer values stored in character variables. For example:

```
#include <iostream>
using namespace std;

char value;

int main() {
  value = 97;
  value = value + 3;
  cout << "The character is " << value << endl;
}
```

What does this program display on the screen? Note that special characters are also characters.
So we could have this line in a program:

```
  k = '\n';
```

How to input characters

```
#include <iostream>
using namespace std;

char c;
int code;

int main() {
  cout << "Press a key followed by Enter ";
  cin >> c;
  code = c;
  cout << "\nThe character is " << c << endl;
  cout << "\nThe character " << c << " has ASCII code " << code;
}
```

This code works fine most of the time. But it doesn't work for certain characters such as <space>.

NOTE – the next program is NOT part of our course. It is only for those who want to hack deeper. This program uses the function `getchar` (from the `cstdio` library) to read a wider range of characters.

```
#include <iostream>
#include <cstdio>
using namespace std;

char c;
int code;

int main() {
  cout << "\n\nPress a key followed by Enter ";
  c = getchar();
  getchar();       // to flush the enter from the input buffer
  code = c;
  cout << "\nThe character is " << c << endl;
  cout << "\nThe character " << c << " has ASCII code " << code;
}
```

Definitions – digit, letter and punctuation

digit – a digit is one of the characters '0' to '9'. Any number (in the decimal number system) is made up of a series of digits.

letter – a letter is one of the characters 'a' to 'z' and also 'A' to 'Z'. Any word (in English) is made up of a series of letters. Lower case letters are 'a' to 'z' and upper case letters are 'A' to 'Z'. Upper case letters are also known as capital letters (or caps).

punctuation – a set of characters that are used together with letters to form English sentences. They include comma, dash (or hyphen), full stop (or period), exclamation mark, question mark, semicolon, colon, brackets, apostrophe and quotation marks.

Example: How to convert a lower case letter to an upper case letter:

```
#include <iostream>
using namespace std;

char letter;

int main() {
  cout << "Press a key followed by Enter ";
  cin >> letter;
  if ((letter >= 'a') && (letter <= 'z')) {
    letter = letter - 32;
    cout << "The upper case letter is " << letter << endl;
  }
}
```

Note that we convert lower case to upper case by subtracting 32. Study the ASCII codes to see why.

What will happen if you type a character that is not a lower case letter, for example `'A'` or `'+'`?

Modify the program to display an appropriate message in this case.

Change this program to convert upper case letters to lower case.

Example: How to repeat a program with a question:

```
#include <iostream>
using namespace std;

int a, b, result;
char choice;

int main() {
  choice = 'X';    // any character that is not Q
  while ((choice != 'Q') && (choice != 'q')) {
    cout << "Enter two numbers ";
    cin >> a >> b;
    result = a + b;
    cout << "The result is " << result << endl;
    cout << "Enter Q to quit - any other key to continue";
    cin >> choice;
  }
}
```

Interesting things to notice in this program:

- the program adds two numbers – the other instructions are to repeat the program
- we realise that a person may press `'Q'` or `'q'` so we cover both of these situations
- work through and understand the test `((choice != 'Q') && (choice != 'q'))`

Arrays of characters

We can create and use arrays of characters, for example:

```
#include <iostream>
using namespace std;

char data[63];

int main() {
  data[34] = 'h';
  data[57] = 's';
  cout << data[34] << " " << data[57] << endl;
}
```

Strings

We often want to use more than just a single characters. We want to work with names, words and sentences. For this we use a string.

Example: A program that uses a string

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  msg = "Good evening to you";
  cout << msg << endl;
}
```

Interesting things to notice in this program:

- you need to #include the `string` library to work with strings
- place double quotes around the text

Example: A program that reads a string (input)

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  cout << "Enter your text message: ";
  cin >> msg;
  cout << msg << endl;
}
```

Problem: The input stops when it gets to a space

The program above only works if you enter one word with no spaces. What happens if you enter a message consisting of several words with spaces between them? To be able to read in a line of several words we use the `getline` function from the `string` library.

```
#include <iostream>
#include <string>
using namespace std;

string msg;

int main() {
  cout << "Enter your text message: ";
  getline(cin, msg);
  cout << msg << endl;
}
```

Remember:

`cin` reads one word at a time (stops at a space)
`getline` reads in one line at a time (can include any number of spaces)

Problems with the input buffer

A buffer is a temporary storage area. The input buffer stores any key stroke you make and then passes it on to your program when the program executes an input instruction. Something to tend to forget is that when we enter numeric data, we type a number and then press enter. The enter is stored in the input buffer and passed to the program at the next input instruction. This can cause problems as shown in the next example program.

```
#include <iostream>
#include <string>
using namespace std;

string name;
int age;

int main() {
  cout << "Enter your age: ";
  cin >> age;
  cout << "Enter your name: ";
  getline(cin, name);
  cout << "Your name is " << name << " and your age is " << age;
}
```

Try this program – it does not produce the correct output. This is because the stored enter (after the age) is passed through to the `getline` function and so the name becomes enter. We need to ***flush the buffer*** – clear the enter out of the buffer so that we can enter the name. We can add `getchar` after reading the age – this will read the stored enter and leave the buffer empty and ready to read the name. Remember that you need to #include `cstdio` to be able to use `getchar`

```
#include <iostream>
#include <cstdio>
#include <string>
using namespace std;

string name;
int age;

int main() {
  cout << "Enter your age: ";
  cin >> age;
  getchar();  // this reads the enter and flushes the buffer
  cout << "Enter your name: ";
  getline(cin, name);
  cout << "Your name is " << name << " and your age is " << age;
}
```

A string is an array of characters

A string is an array of characters. Like any array, you can use the index number to access a particular element of the array.

Example: A program that changes a particular character in a string

```
#include <iostream>
#include <string>
using namespace std;

string text;

int main() {
  text = "We are going to sleep";
  cout << text << endl;
  text[17] = 'w';
  cout << text << endl;
}
```

Interesting things to notice in this program:

- just one character is changed
- you need to calculate what the index number is – start from zero and include spaces

Example program: convert characters in a string to a number

Enter a street address, for example, 36 Ocean View Road. It is impossible to know how many spaces there are going to be in the address. Check if the number is less than or greater than 500 (couriers need to do this sort of thing).

Possible solution – read the whole address as a string. Convert from characters in a string to a number. Formula – get a digit and convert from character to number by subtracting 48 (because the ASCII code of zero is 48). Then multiply the number by 10 and add the new digit.

This program looks like this:

```
#include <iostream>
#include <string>
using namespace std;

string input;
int num, i;

int main() {
  cout << "Enter a street address, e.g. 36 Ocean View Road\n";
  getline(cin, input);
  i = 0;
  num = 0;
  while (input[i] != ' ') {
    num = num * 10 + input[i] - 48;
    i++;
  }
  if (num > 500) {
    cout << "This address is greater than 500";
  } else {
    cout << "This address is less than or equal to 500";
  }
}
```

Interesting things to notice in this program:

- the loop continues until a space has been found – the space after the number
- use the formula to convert characters to numbers – useful to remember this
- there are many ways to write a program – there will be alternatives

Concatenating strings

Concatenating two strings means placing the strings side by side to create a larger string. For example, if string1 is "Hello" and string2 is "there" and we concatenate them, we would get "Hellothere". In C++ we use the plus sign to do concatenation of strings.

```
#include <iostream>
#include <string>
using namespace std;

string string1, string2, string3;

int main() {
  string1 = "How are";
  string2 = "you today?";
  string3 = string1 + string2;
  cout << string3 << endl;
}
```

Remember:

With numbers, the plus sign is used for **addition**.
With strings, the plus sign is used for **concatenation**.

Example : reverse a string

A program to reverse a string:

```
#include <iostream>
#include <string>
using namespace std;

int i, total;
string sentence, reverse;

int main() {
  cout << "Type in a sentence ";
  getline(cin, sentence);
  total = sentence.length();
  reverse = "";
  for (i = total - 1; i >= 0; i--) {   // note the i--
    reverse = reverse + sentence[i];
  }
  cout << "The sentence is " << sentence << endl;
  cout << "The reverse is  " << reverse << endl;
}
```

Does this program work correctly? Do a ***walkthrough*** to find out.

Here is an example of a walkthrough for this program. Make a list of the variables and then work through the program, one instruction at a time. When a variable changes value, cross it out and write the new value next to the variable name.

```
i = 5  4  3  2  1  0  -1

j = 0  1  2  3  4  5  6

total = 6

sentence = My cat

reverse = tac yM
```

We also keep track of what is displayed on the screen, as follows:

```
Type in a sentence My cat
The sentence is My cat
The reverse is  tac yM
```

Example: replace a word

Replace a word in a sentence. Here is an example when the program is running:

```
Enter the word to find: bird
Enter the replacement word: fish
Enter a sentence: I saw a bird go by
```

```
Final sentence is: I saw a fish go by
```

And here is the program:

```
#include <iostream>
#include <cstdio>
#include <string>
using namespace std;

string findword, replace, sentence, final, word;
int total, i;

int main() {
  cout << "Enter the word to find: ";
  cin >> findword;
  getchar();
  cout << "Enter the replacement word: ";
  cin >> replace;
  getchar();
  cout << "Enter a sentence: ";
  getline(cin, sentence);
  total = sentence.length();
  final = "";
  word = "";
  for (i = 0; i < total; i++) {
    if (sentence[i] == ' ') {
      if (word == findword) {
        final = final + replace + " ";
      } else {
        final = final + word + " ";
      }
      word = "";
    } else {
      word = word + sentence[i];
    }
  }
  if (word == findword) {
    final = final + replace + " ";
  } else {
    final = final + word + " ";
  }
  cout << "Final sentence is: " << final << endl;
}
```

Interesting things to notice in this program:

- note the useful function `length()` that tells you how many characters in the string
- build up a word by looking for spaces in the sentence
- build up the final sentence by concatenating words (or replacement words)
- problem – it only works if there are spaces between words and not other characters
- note how you have to check for the final word after the loop has finished